

Reporte Proyecto Final – CodeFlix (Género: Fantasía)

Equipo Turquesa – Ingeniería de Software

INTEGRANTES:

Celaya Nava Carlos Adrián
Del Monte Ortega Maryam Michelle
Flores Arriola Edson Rafael
Martínez Mejía Eduardo
Monroy Romero Sahara Mariel

1. Visión General del Proyecto:

Descripción:

Desarrollo de un sitio web temático de cine centrado en el género de fantasía. El sistema permitirá consultar un catálogo de películas, visualizar detalles, filtrar por subgénero, registrar usuarios, iniciar sesión y gestionar listas de favoritos.

Objetivo principal:

Aplicar los conceptos de Ingeniería de Software: planificación ágil, modelado UML, análisis de requisitos, arquitectura, desarrollo web, pruebas y presentación técnica.

Tecnologías elegidas:

- **Frontend:** React, con TailwindCSS.
- **Backend:** [Node.js](#).
- **Base de datos:** MySQL, API TMDB.
- **Control de versiones:** Git + GitHub.
- **Tablero ágil:** Trello.

2. Historias de usuario:

- **US01:** Como visitante quiero ver el catálogo de películas para poder descubrir nuevas obras.
- **US02:** Como visitante quiero ver detalles de una película para saber si me interesa verla.
- **US03:** Como visitante quiero poder encontrar una película mediante un buscador para poder ver detalles sobre ella.
- **US04:** Como usuario quisiera que la página me recomiende películas que me puedan interesar.
- **US05:** Como usuario quiero registrarme en el sistema para poder aprovechar todo lo que este ofrece.
- **US06:** Como usuario quiero iniciar sesión para acceder a mi cuenta.
- **US07:** Como usuario quiero que el catálogo pueda filtrarse por diferentes características como subgénero, año, idioma, etc.
- **US08:** Como usuario quiero poder ver los detalles y datos de cada película que seleccione.
- **US09:** Como usuario quiero poder ver imágenes de la película para poder decidir mejor si me interesa.
- **US10:** Como usuario, quisiera que la plataforma tenga una estética fantasiosa para sentir más inmersión a un mundo diferente.

3. Requisitos funcionales:

- El sistema debe mostrar el catálogo de películas y los detalles, que incluyen: título, descripción, rating, año.
- El catálogo debe tener paginación o scroll para ver las películas.
- El catálogo debe tener secciones dentro del género de Fantasía: Mejor calificadas, más recientes, más populares, recomendadas.
- El sistema debe ofrecer una barra de búsqueda por título de película
 - Debe mostrar resultados en una lista con el mismo formato de tarjetas del catálogo.
 - Debe mostrar sugerencias conforme se vaya escribiendo en la barra.
 - Debe mostrar un mensaje de “Sin resultados” cuando no haya coincidencias.
- Debe existir un endpoint o servicio que devuelva recomendaciones por usuario.
- Las recomendaciones deben mostrarse en la landing (si usuario está autenticado) y en la ficha de detalle como “Te puede interesar”.
- El sistema debe permitir iniciar sesión con email y contraseña.
- El sistema debe mantener sesión mediante token seguro
- Debe permitir cerrar sesión desde el menú de usuario.

3. Requisitos no funcionales:

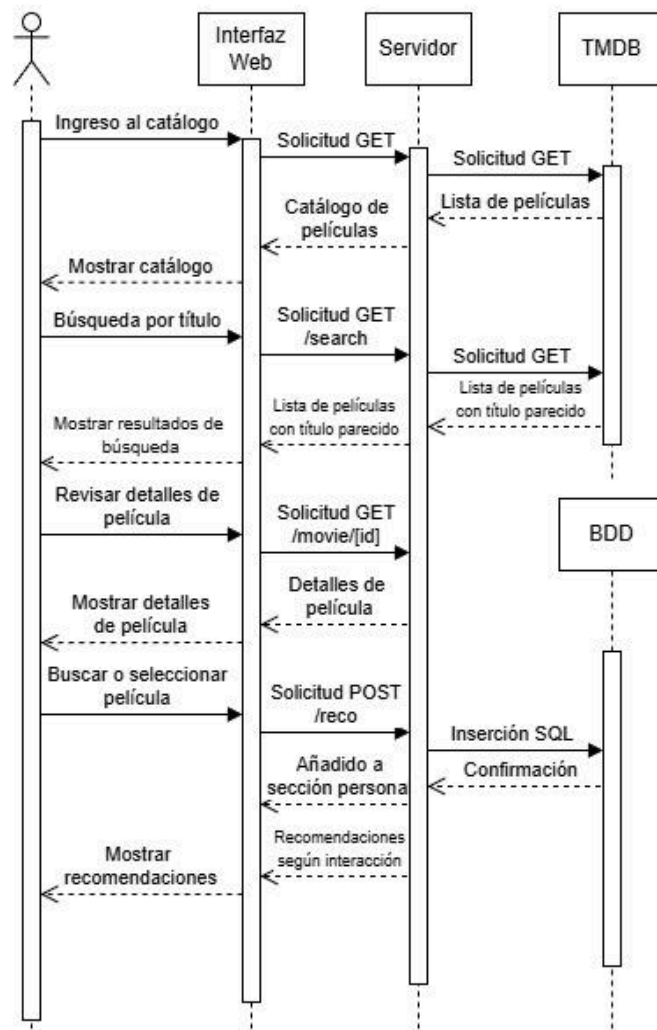
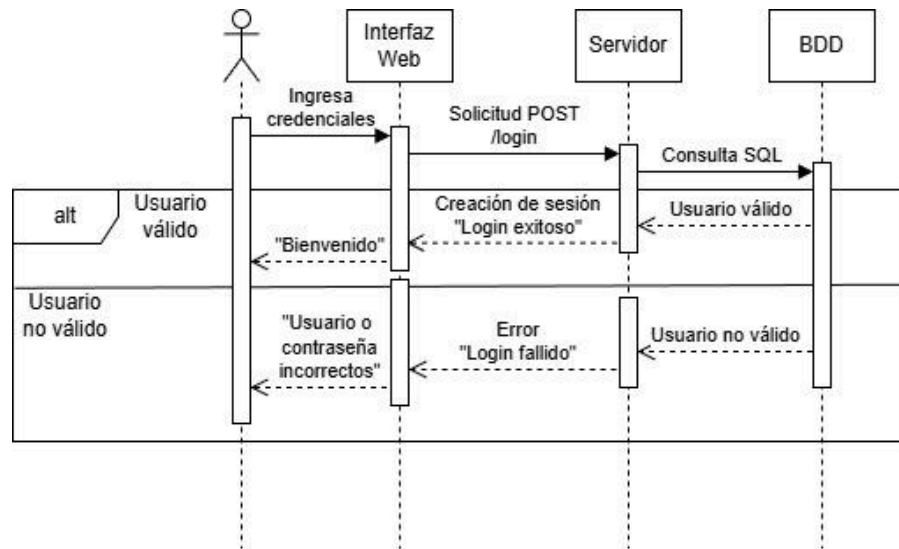
- El sistema debe verse correctamente en pantallas de escritorio, tablet y móvil (layout responsive).
- El sistema no debe tardar demasiado
- El sistema debe cumplir con ser color turquesa
- La interfaz debe ser intuitiva, con navegación clara y textos legibles.
- El código debe estar organizado en capas (frontend y backend separados) y versionado en GitHub con mensajes de commit descriptivos.
- El sistema no debe exponer credenciales en el repositorio y debe validar datos de entrada básicos
- Endpoints REST bien versionados

4. Requisitos opcionales:

- Almacenamiento seguro de contraseñas: Hash con bcrypt/argon2
- Gestión de sesión: Tokens con expiración.



❖ Diagrama de secuencia:



6. Informe de desarrollo:

Semana 1 (17 - 21 de noviembre):

- Reunión por videoconferencia vía Meet para concretar planificación, metodología y herramientas a utilizar.
- Obtención de requisitos mediante *user stories*, trazado de diagramas UML y repartición de tareas.
- Comunicación asíncrona vía WhatsApp para consultas o ayuda entre los integrantes del equipo.

Semana 2 (24 - 28 de noviembre):

- Comunicación asíncrona vía WhatsApp para consultas y avisos sobre progreso en el proyecto.
- Creación del sitio utilizando NextJS.
- Implementación del catálogo de películas en distintos apartados: 'Más populares', 'Más recientes', 'Mejor calificadas', 'Próximos estrenos' y 'Recomendadas'
- Implementación de un apartado para visualizar detalles sobre cada película (breve sinopsis, imágenes, reparto y director).
- Implementación de un sistema de inicio de sesión.
- Implementación de barra de búsqueda por título.
- Creación del reporte/informe de desarrollo del proyecto.

Fin de semana previo a la entrega (29 y 30 de noviembre):

- Creación de pruebas unitarias para los archivos que conforman el proyecto.
- Documentación.
- Limpieza y refactorización de código.
- Correcciones y mejoras en sistema de login, búsqueda y recomendaciones.

7. Arquitectura cliente servidor y API

Optamos por una arquitectura cliente-servidor moderna utilizando Next.js por varias razones fundamentales. Next.js nos permitió implementar un enfoque híbrido que combina las ventajas del renderizado del lado del servidor (SSR) para mejorar el SEO y el rendimiento inicial de carga, con la interactividad de una aplicación de una sola página (SPA). Esto es crucial para un catálogo de películas donde necesitábamos que las páginas de detalles fueran indexables por motores de búsqueda, mientras manteníamos una experiencia de usuario fluida al navegar entre categorías.

Por otro lado, la elección de The Movie Database (TMDB) como fuente de datos nos proporcionó una API robusta y actualizada con información completa sobre películas del género fantasía. Implementamos una capa de abstracción en nuestro servidor Next.js para gestionar las solicitudes a TMDB, cachear respuestas cuando era apropiado, y transformar los datos a un formato óptimo para nuestro frontend. Esto nos permitió mantener una separación clara de responsabilidades y asegurar que los cambios en la API de TMDB no afectaran directamente a nuestra interfaz de usuario.

8. Estructura del Proyecto y Mantenibilidad

Organizamos el código siguiendo una estructura modular escalable que facilita el mantenimiento y la adición de nuevas funcionalidades. Separamos claramente los componentes de UI, las utilidades de fetch, los hooks personalizados y las páginas. Esta organización, combinada con TypeScript para tipado estático, redujo significativamente los errores en tiempo de desarrollo y mejoró la colaboración entre miembros del equipo.

9. Rendimiento y Experiencia de Usuario

Implementamos estrategias de renderizado optimizadas como generación estática para páginas que no cambian frecuentemente y renderizado del lado del servidor para contenido dinámico. Además, añadimos características como búsqueda en tiempo real y filtrado por criterios que se ejecutan principalmente en el cliente para minimizar las solicitudes al servidor, proporcionando una experiencia responsive que mantiene al usuario interesado en películas de fantasía.

10. Conclusiones

En este proyecto aprendimos que documentar desde el primer día aunque sea de forma básica, usar control de versiones con commits descriptivos, y mantener una estructura de proyecto clara, nos salvó incontables horas cuando tuvimos que añadir nuevas features o depurar errores.

También nos dimos cuenta de que seguir una metodología ágil flexible con sprints cortos y reuniones breves esporádicas fue clave. No se trataba solo de cumplir plazos, sino de adaptarnos rápido cuando descubríamos que cierta funcionalidad no era tan útil para el

usuario. La parte más valiosa fue aprender que el buen software se construye iterando, colaborando y manteniendo siempre la simplicidad como guía.